

Answer the questions in the boxes provided on the question sheets. If you run out of room for an answer, add a page to the end of the document.

Related Readings:

- Proof Primer: <https://canvas.wisc.edu/courses/485480/files/50373320>
- CS240 Readings: <http://pages.cs.wisc.edu/~hasti/cs240/readings/>

Name: _____

Wisc id: _____

Induction

1. Prove the following by induction.

(a) $\sum_{i=1}^n i = \frac{n(n+1)}{2}$

Solution:

Base case: when $n = 1$; $\frac{n(n+1)}{2} = 1$

Assume $\sum_{i=1}^k i = \frac{k(k+1)}{2}$ for some $k \in \mathbb{N}$ (inductive hypothesis)

Induction step for $k + 1$:

$$\begin{aligned} \sum_{i=1}^{k+1} i &= \left(\sum_{i=1}^k i \right) + k + 1 \\ &= \frac{k(k+1)}{2} + k + 1 && \text{(by inductive hypothesis),} \\ &= \frac{(k(k+1) + 2k + 2)}{2} = \frac{(k+1)(k+2)}{2} \end{aligned}$$

(b) $\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$

Solution:

Base case: when $n = 1$; $\frac{n(n+1)(2n+1)}{6} = 1$

Assume $\sum_{i=1}^k i^2 = \frac{k(k+1)(2k+1)}{6}$ for some $k \in \mathbb{N}$ (inductive hypothesis).

Induction step for $k + 1$:

$$\begin{aligned} \sum_{i=1}^{k+1} i^2 &= \left(\sum_{i=1}^k i^2 \right) + (k+1)^2 \\ &= \frac{k(k+1)(2k+1)}{6} + (k+1)^2 && \text{(by inductive hypothesis),} \\ &= \frac{(k(k+1)(2k+1) + 6(k+1)^2)}{6} \\ &= \frac{(k+1)(2k^2 + 7k + 6)}{6} \\ &= \frac{(k+1)(k+2)(2k+3)}{6}. \end{aligned}$$

(c) $\sum_{i=1}^n i^3 = \frac{n^2(n+1)^2}{4}$

Solution:

Base case: when $n = 1$; $\frac{n^2(n+1)^2}{4} = 1$

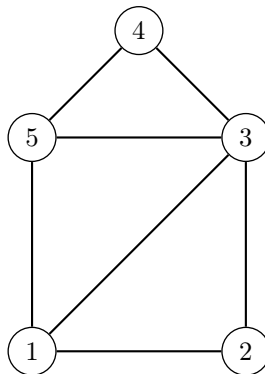
Assume $\sum_{i=1}^k i^3 = \frac{k^2(k+1)^2}{4}$ for some $k \in \mathbb{N}$ (inductive hypothesis)

Induction step for $k + 1$:

$$\begin{aligned}
 \sum_{i=1}^{k+1} i^3 &= \left(\sum_{i=1}^k i^3 \right) + (k+1)^3 \\
 &= \frac{k^2(k+1)^2}{4} + (k+1)^3 && \text{by inductive hypothesis,} \\
 &= \frac{(k^2(k+1)^2 + 4(k+1)^3)}{4} \\
 &= \frac{(k+1)^2(k^2 + 4k + 4)}{4} \\
 &= \frac{(k+1)^2(k+2)^2}{4}.
 \end{aligned}$$

Graphs and Trees

2. Give the adjacency matrix, adjacency list, edge list, and incidence matrix for the following graph.



Solution: Adjacency Matrix:
$$\begin{pmatrix} 0 & 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 & 0 \end{pmatrix}$$

Adjacency List:
$$\begin{pmatrix} (2, 3, 5) \\ (1, 3) \\ (1, 2, 4, 5) \\ (3, 5) \\ (1, 3, 4) \end{pmatrix}$$

Edge List: $((1, 2), (1, 3), (1, 5), (2, 3), (3, 4), (3, 5), (4, 5))$

Incidence Matrix:
$$\begin{pmatrix} 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 1 & 1 \end{pmatrix}$$

3. How many edges are there in a complete graph of n vertices? Prove by induction.

Solution: Let K_n denote the complete graph on n vertices. We will show that K_n has $\frac{n(n-1)}{2}$ edges by induction on the number of vertices.

Base case: when $n = 1$, K_1 is a single vertex and has 0 edges, and $\frac{1(1-1)}{2} = 0$.

Suppose K_m has $m(m-1)/2$ edges for some $m \in \mathbb{N}$.

We would like to show that K_{m+1} has $\frac{(m+1)(m+1-1)}{2} = \frac{(m+1) \cdot m}{2}$ edges.

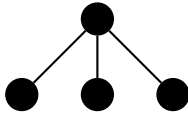
Given K_{m+1} , we may remove one vertex v and all the m many edges attached to it. This will result in a graph $G = (V, E)$ on m vertices.

Notice that for every $u, w \in V$, $(u, w) \in E$ because u, w are also vertices of K_{m+1} and, K_{m+1} is assumed to be complete. Therefore, $G = K_m$, and thus $|E| = \frac{m \cdot (m-1)}{2}$ (by inductive hypothesis).

When obtaining G from K_{m+1} , we removed m edges, so the number of edges of $K_{m+1} = m + \frac{m(m-1)}{2} = \frac{m}{2}(2 + m - 1) = \frac{(m+1) \cdot m}{2}$ as desired.

4. Draw all possible (unlabelled) trees with 4 nodes.

Solution:



5. Show by induction that, for all trees, $|E| = |V| - 1$.

Solution: We prove by strong induction on the number of vertices.

Let $T = (V, E)$ be a tree with one vertex. Then, there is no edge, so $|E| = 0 = 1 - 1 = |V| - 1$.

Suppose every tree with j vertices has $j - 1$ edges for $1 \leq j \leq k$ where $k \in \mathbb{N}$.

Let $T = (V, E)$ be a tree with $|V| = k + 1$. Consider an arbitrary vertex v . Let $d = \deg(v)$, i.e, the number of edges attached to v . Consider the graph G obtained by removing v from T . G must have d connected components, $G_1 = (V_1, E_1), \dots, G_d = (V_d, E_d)$ and they must also all be trees. This is because T has no cycle, so there cannot be a path between the neighbors of v . And of course, there cannot be a cycle within each G_i .

By the hypothesis, since $1 \leq |V_i| \leq k$, we have $|E_i| = |V_i| - 1$ for all $1 \leq i \leq d$. The total number of vertices in T is then $1 + \sum_{i=1}^d |V_i|$ where the 1 comes from including v . The total number of edges in T is $d + \sum_{i=1}^d |E_i|$ where d comes from the edges attached to v . We have

$$|E| = d + \sum_{i=1}^d |E_i| = d + \sum_{i=1}^d (|V_i| - 1) = \sum_{i=1}^d |V_i| = (1 + \sum_{i=1}^d |V_i|) - 1 = |V| - 1$$

as desired.

Alternatively, one can use the fact that every tree has a leaf (see discussion 1) and claim that one can always remove the leaf and one edge to obtain a tree with one fewer vertex.

Proofs

6. Show that $2x$ is even for all $x \in \mathbb{N}$.

(a) By direct proof.

Solution: By definition, an even number is divisible by 2, and $2x$ is divisible by 2. □

(b) By contradiction.

Solution: Assume to the contrary that $2x$ is odd. This implies that $2x$ is not perfectly divisible by 2. However, $\frac{2x}{2} = x$, a contradiction.

7. For all $x, y \in \mathbb{R}$, show that $|x + y| \leq |x| + |y|$. (Hint: use proof by cases.)

Solution:

Case 1: $x \geq 0, y \geq 0$. $|x + y| = x + y = |x| + |y|$.

Case 2: $x < 0, y < 0$. $|x + y| = -(x + y) = |x| + |y|$.

Case 3: $x = 0, y < 0$. $|x + y| = |y| < |x| + |y|$.

Case 4: $x > 0, y < 0$ and $|x| \geq |y|$. $|x + y| < |x| \leq |x| + |y|$.

Case 5: $x > 0, y < 0$ and $|x| < |y|$. $|x + y| < |y| \leq |x| + |y|$.

WLOG: Case 3 covers $x < 0, y = 0$; Case 4 covers $y > 0, x < 0$ and $|y| \geq |x|$; and Case 5 covers $y > 0, x < 0$ and $|y| < |x|$

Program Correctness (and Invariants)

8. For the following algorithms, describe the loop invariant(s) and prove that they are sound and complete.

Algorithm 1: findMin

(a) **Input:** a : A non-empty array of integers (indexed starting at 1)
 Output: The smallest element in the array
 begin
 $min \leftarrow \infty$
 for $i \leftarrow 1$ **to** $\text{len}(a)$ **do**
 if $a[i] < min$ **then**
 $min \leftarrow a[i]$
 end
 end
 return min
 end

Solution:

Invariant: At the end of step i of the ‘for’ loop, min contains the minimum value from index 1 to i of a .

Proof: We prove the above invariant by induction on the number of iterations of the ‘for’ loop.

After the first iteration, min is set to $a[1]$.

Assume that min is minimum value of $a[1, \dots, k]$ after the k th iteration.

After the $(k + 1)^{th}$ iteration, min will be the minimum of min and $a[k + 1]$. Hence, min is the minimum of $a[1, \dots, k + 1]$.

Soundness: From the loop invariant, after completing the step equal to the length of the input array, min will contain the minimum element.

Completeness: With each iteration of the loop, the counter i moves one step closer to the end of the input array, and the loop (and the algorithm) terminates after reaching the end of the array.

Algorithm 2: InsertionSort

(b)

```

Input:  $a$ : A non-empty array of integers (indexed starting at 1)
Output:  $a$ , sorted from largest to smallest
begin
  for  $i \leftarrow 2$  to  $\text{len}(a)$  do
     $val \leftarrow a[i]$ 
    for  $j \leftarrow 1$  to  $i - 1$  do
      if  $val > a[j]$  then
        shift  $a[j..i - 1]$  to  $a[j + 1..i]$ 
         $a[j] \leftarrow val$ 
        break
      end
    end
  end
  return  $a$ 
end

```

Solution:

Invariant: At the end of i^{th} iteration of the outer ‘for’ loop, the items $a[1..i]$ are sorted from largest to smallest.

Proof: We prove the above invariant by induction on the number of iterations of the outer ‘for’ loop.

After the first iteration, the item $a[1]$ is sorted.

Assume that $a[1, \dots, k]$ is sorted after the k^{th} iteration for some $k \in \mathbb{N}$.

After the $(k + 1)^{\text{th}}$ iteration, $a[k + 1]$ will be placed at an index ‘ j ’ such that the items in $a[1, \dots, j - 1]$ are sorted and larger than $a[k + 1]$. The sorted $a[j, k]$ items less than (or) equal to $a[k + 1]$ are shifted to $a[j + 1, k + 1]$. Hence, $a[1, \dots, k + 1]$ is sorted.

Soundness: From the loop invariant, after completing the iteration (of the outer ‘for’ loop) equal to the length of the input array, a will be sorted from largest to smallest.

Completeness: With each iteration of the outer ‘for’ loop, the outer loop counter i moves one step closer to the end of the input array, and the outer loop (and the algorithm) terminates after reaching the end of the array.

For each iteration of the outer ‘for’ loop, the inner ‘for’ loop increments by 1 from 1 to i (the counter of the outer loop), and this ensures that the inner loop will always terminate.

Coding Question: Hello World

Most assignments will have a coding question. You can code in C, C++, C#, Java, Python, or Rust. You will submit a Makefile and a source code file.

Makefile: In the Makefile, there needs to be a build command and a run command. Below is a sample Makefile for a C++ program. You will find this Makefile in assignment details. Download the sample Makefile and edit it for your chosen programming language and code.

```
#Build commands to copy:
#Replace g++ -o HelloWorld HelloWorld.cpp below with the appropriate command.
#Java:
#    javac source_file.java
#Python:
#    echo "Nothing to compile."
#C#:
#    mcs -out:exec_name source_file.cs
#C:
#    gcc -o exec_name source_file.c
#C++:
#    g++ -o exec_name source_file.cpp
#Rust:
#    rustc source_file.rs

build:
    g++ -o HelloWorld HelloWorld.cpp

#Run commands to copy:
#Replace ./HelloWorld below with the appropriate command.
#Java:
#    java source_file
#Python 3:
#    python3 source_file.py
#C#:
#    mono exec_name
#C/C++:
#    ./exec_name
#Rust:
#    ./source_file

run:
    ./HelloWorld
```

9. HelloWorld Program Details

The input will start with a positive integer, giving the number of instances that follow. For each instance, there will be a string. For each string s , the program should output Hello, s ! on its own line.

A sample input is the following:

```
3
World
Marc
Owen
```

The output for the sample input should be the following:

```
Hello, World!
Hello, Marc!
Hello, Owen!
```